Instacart Market Basket Analysis: Predicting User Reorders

Executive Summary

This project develops a supervised machine learning system to predict whether a customer will reorder a previously purchased product on Instacart. Using over three million grocery transactions, we construct a structured modeling dataset containing behavioral, contextual, and product-level predictors.

Multiple classification models are evaluated, including Logistic Regression, Decision Trees, Random Forest, XGBoost, and LightGBM. Because reorder events represent the minority class, model performance is assessed primarily using Precision-Recall AUC, supplemented by ROC-AUC and the best achievable F1 score across probability thresholds.

The results show that ensemble gradient boosting methods significantly outperform linear models. A hyperparameter-tuned XGBoost model achieves the best performance with a PR-AUC of 0.7960, a ROC-AUC of 0.7293, and a best F1 score of approximately 0.764. Cross-validation confirms that this model provides the most stable predictive performance.

These findings demonstrate that customer purchasing habits can be accurately modeled using historical transaction data. Such predictive systems can power personalized product recommendations, automated shopping lists, and targeted promotions for online grocery platforms.

1 Introduction & Problem Statement

The retail and grocery industries are highly driven by recurring customer behavior. For this project, we use the Instacart Market Basket Analysis dataset, a large-scale repository of over three million grocery orders. The goal is to build a predictive model that can determine whether a user will reorder a previously purchased product in a later order.

While it is common in retail analytics to frame customer behavior tasks as unsupervised learning problems, such approaches often fall short of actionable business impact. Finding clusters is useful only if those clusters affect a concrete business target such as future spend or retention. This report therefore formulates the problem strictly as supervised learning: we predict a known binary response variable, `reordered`, using user history, order timing, and product attributes that are available at prediction time.

2 Data Description: The Modeling Array

A critical component of this supervised learning task is the construction of a flat, two-dimensional modeling array from relational source tables.

The raw data consists of six relational CSV files covering orders, prior-order line items, current-order line items, and product metadata. Orders are chronologically split into "prior" historical data and "train" orders that serve as the current orders for prediction.

After merging product data, historical aggregates, and current-order metadata, we construct a single structured tabular array for classification. Each row corresponds to exactly one `(user_id, product_id)` pair in the context of a specific train order. In other words, each row represents one

product that a user might purchase in the current order, provided the user has interacted with that product previously.

The target variable is `reordered`, where 1 indicates that the user reordered the product in the current order and 0 indicates that they did not.

Table 1: Predictor variables used in the modeling array

Feature	Type	Source	Description
`order_dow`	Context	orders.csv	Day of week when the order was placed
`order_hour_of_day`	Context	orders.csv	Hour of day when the order was placed
`days_since_prior_order`	Context	orders.csv	Days since the user's previous order
`user_order_number`	Context	orders.csv	Sequential order number for the user
`up_order_number`	User-Product History	prior orders	Number of prior orders containing this product
`up_reorder_rate`	User-Product History	prior orders	Fraction of prior orders where this product was reordered
`user_reorder_rate`	User History	prior orders	Overall reorder rate for the user
`aisle`	Product Attribute	aisles.csv	Product aisle category
`department`	Product Attribute	departments.csv	Product department category

3 Data Preprocessing & Feature Engineering

Substantial effort is dedicated to preprocessing the relational data into the final modeling array. Raw identifiers are joined to recover categorical labels, and historical behavior is aggregated through grouped transformations.

3.1 Rationale for Predictor Selection

Predictors were selected based on the hypothesis that grocery purchasing behavior is highly habitual and influenced by both historical purchasing patterns and contextual factors. Exploratory analysis supports this assumption.

As illustrated in Figure 1, reorder probability varies systematically across several behavioral dimensions. Reorder rates show mild variation across days of the week, while stronger patterns appear across the hour of day, with peaks during early morning shopping hours. Reorder probability also decreases as the number of days since the previous order increases, suggesting that recent orders are more likely to contain repeated products. Finally, users with higher historical reorder rates exhibit substantially higher probabilities of reordering items again. Supplemental target and numeric distribution panels are provided in Appendix A.

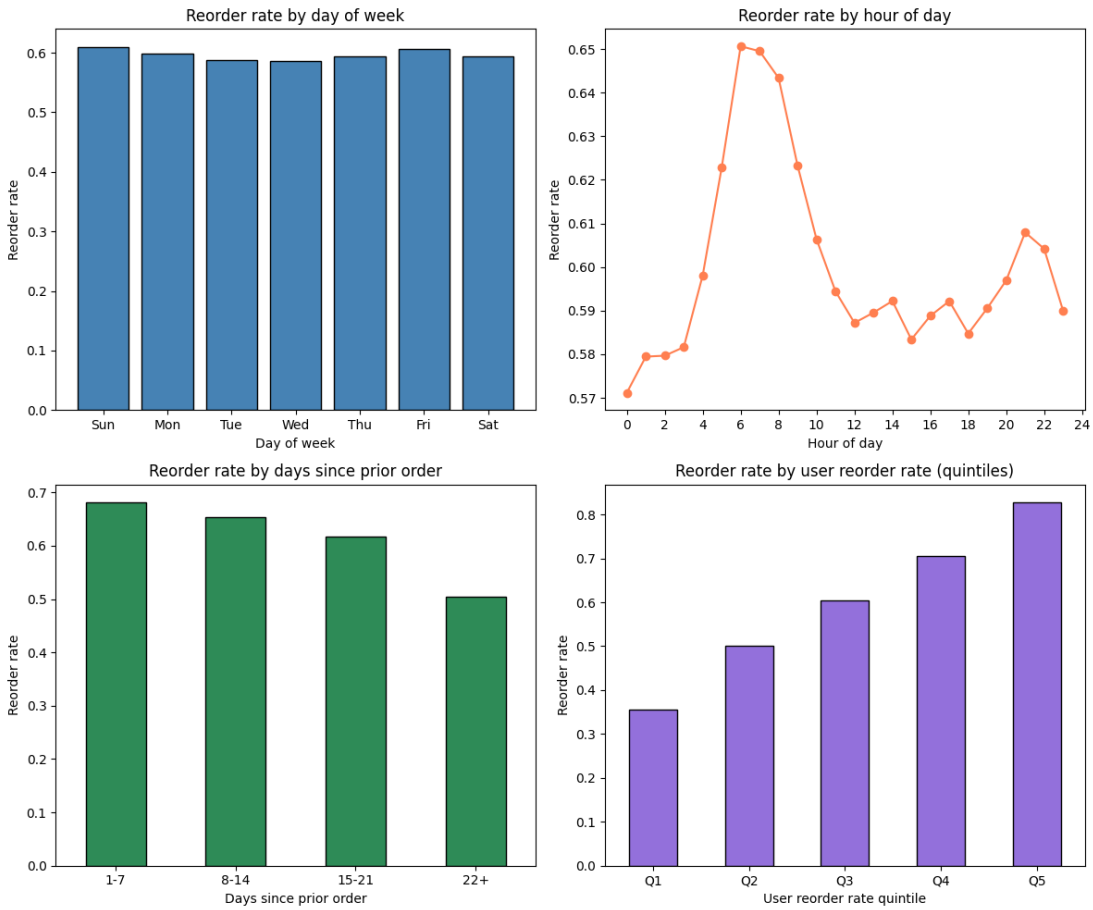

Figure 1: Reorder probability across behavioral factors. Reorder rates vary with time of day, recency of orders, and user-level reorder behavior, indicating strong habitual purchasing patterns.

These empirical observations motivate several categories of predictors:

- **User-Product History** (`up_order_number`, `up_reorder_rate`): Past behavior on a specific item is the strongest indicator of future behavior.
- **Contextual Features** (`order_dow`, `order_hour_of_day`, `days_since_prior_order`): Timing often dictates need, especially for replenishment behavior.
- **Product Attributes** (`aisle`, `department`): Certain categories naturally exhibit higher turnover and reorder rates than others.

3.2 Handling Missing Data

When creating features from historical prior orders, missing values naturally occur for the user-product history fields that track prior interaction count and reorder frequency when a user interacts with a product for the first time. Dropping these rows would discard valuable data. Zero is also not a valid imputation because it is a legitimate reorder rate for users who bought an item once but never reordered it. We therefore impute these missing values with a sentinel value of `-9`. This

ensures that downstream models treat first-time interactions as a distinct, isolated region of the feature space.

4 Supervised Learning Methodology & New Methods

The predictive modeling phase is designed around a rigorous comparative framework so that improvements in reorder prediction are mathematically sound and generalizable.

To ensure a fair comparison across algorithms of varying complexity, all models share the same train/test split. Each model is trained on the same historical data and evaluated on the same holdout set. Final model selection then expands to a 5-fold cross-validation setup to improve stability.

4.1 Model Progression & Standard Methods

1. **Baseline Logistic Regression**: The modeling sequence begins with a simple linear classifier. This baseline establishes that the engineered features contain genuine predictive signal and provides a performance floor for comparison.

2. **Classification Tree**: A standard decision tree introduces nonlinearity and interpretable hierarchical splits in user behavior.

3. **Random Forest**: Bagging multiple trees helps reduce the variance associated with a single tree and improves generalization.

4. **XGBoost**: Sequential gradient boosting captures more complex nonlinear shopping patterns by explicitly correcting residual errors from earlier trees.

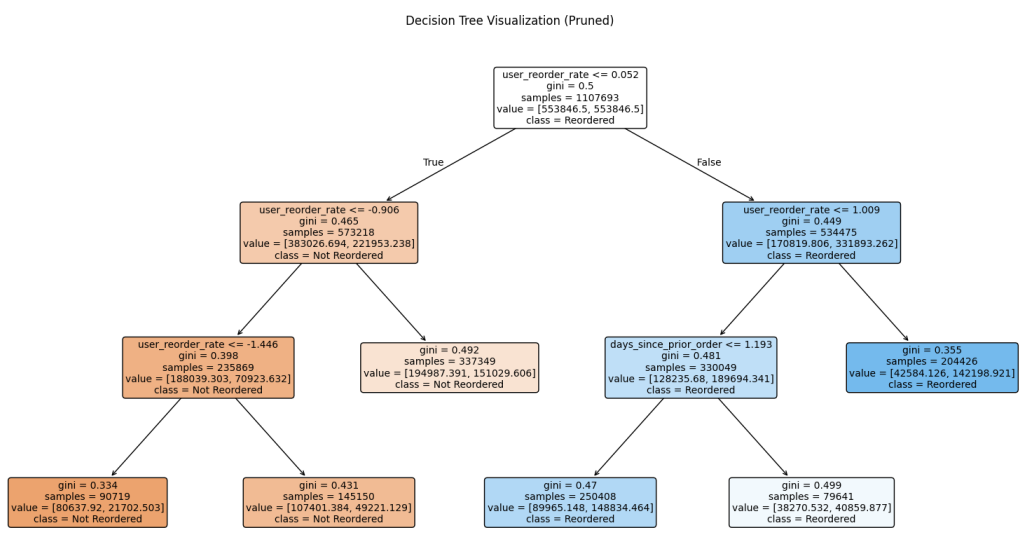

Figure 2: Pruned decision tree trained on the dataset. The first split occurs on user-level reorder behavior, and later splits incorporate contextual factors such as days since the prior order. Additional feature-importance diagnostics are reported in Appendix B.

4.2 Implementation of New Methods

To explore methods beyond the standard class toolkit, two advanced approaches were integrated:

- **LightGBM**: While XGBoost grows trees level-wise, LightGBM grows trees leaf-wise. This makes it attractive for the Instacart modeling array because it often trains faster, consumes less memory, and can still achieve competitive predictive accuracy on sparse data.
- **Hyperopt Tuning**: Instead of relying on expensive grid search, Hyperopt performs Bayesian optimization. It learns from previous trials and searches for improved combinations of hyperparameters such as learning rate and tree depth for the boosting ensembles.

5 Results & Model Evaluation

The dataset exhibits substantial class imbalance because users typically do not reorder most available products during a single shopping session. Traditional accuracy metrics are therefore insufficient for evaluating model quality. We instead emphasize:

- **ROC-AUC**: The model's ability to distinguish reordered from non-reordered items.
- **PR-AUC**: Precision-Recall AUC, which focuses on the positive class and is better aligned with imbalanced datasets.
- **Best F1 Score**: The harmonic mean of precision and recall at the best decision threshold.

Table 2 compares all evaluated models, while Figure 3 visualizes the cross-validated trade-offs across the three metrics. Additional evaluation diagnostics are provided in Appendix C.

Table 2: Model performance comparison

Model	ROC-AUC	PR-AUC	Best F1
Logistic Regression	0.7039	0.7714	0.7588
Classification Tree	0.6919	0.7349	0.7582
Random Forest	0.7158	0.7830	0.7618
XGBoost	0.7114	0.7797	0.7606
LightGBM	0.7125	0.7810	0.7609
Tuned XGBoost	**0.7293**	**0.7960**	**0.7640**

The baseline Logistic Regression model demonstrates that the engineered features contain meaningful predictive signal. However, its linear structure limits its ability to capture complex behavioral patterns. Tree-based models introduce nonlinear decision boundaries and improve performance relative to the linear baseline. Random Forest improves both ROC-AUC and PR-AUC over Logistic Regression by reducing variance through ensemble bagging.

Boosting algorithms further enhance predictive power by sequentially correcting errors from previous trees. Among all models, the hyperparameter-tuned XGBoost model achieves the strongest performance, reaching a ROC-AUC of 0.7293, a PR-AUC of 0.7960, and a best F1 score of approximately 0.764.

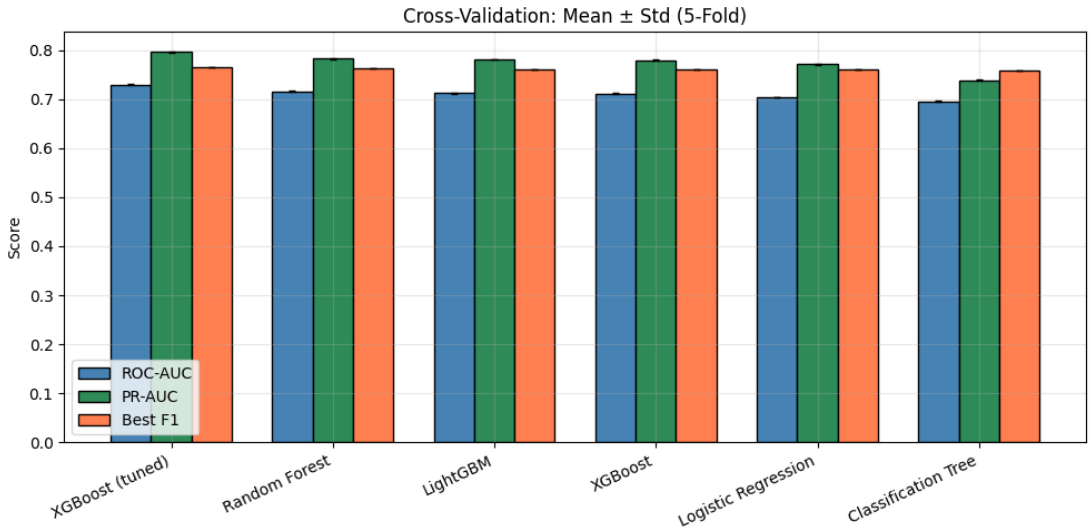

Figure 3: Comparison of ROC-AUC, PR-AUC, and best F1 scores across classification models under 5-fold cross-validation.

These results confirm that gradient boosting methods are particularly effective for modeling user reorder behavior because they capture complex interactions among historical purchase patterns, contextual ordering behavior, and product characteristics. In practice, tuned XGBoost provides the most reliable balance between precision and recall for recommending products that a user is genuinely likely to reorder.

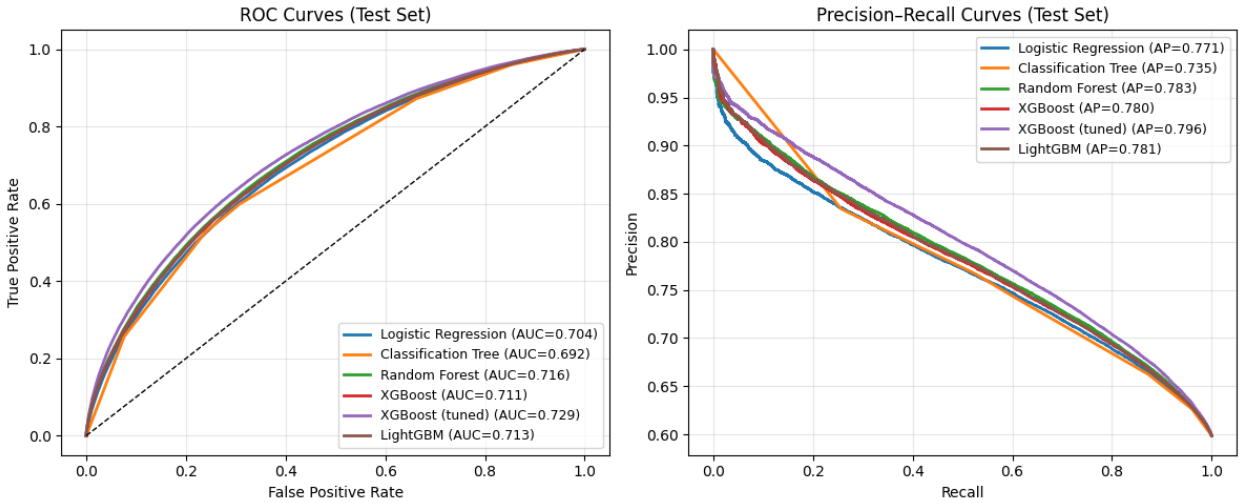

Figure 4: Performance curves for all evaluated models. Because user reorders represent a minority class, the Precision-Recall curve is especially informative for ranking models on this task.

The ROC and Precision-Recall curves in Figure 4 show that tuned XGBoost maintains a higher precision floor than the other models as recall increases. In contrast, the Logistic Regression baseline exhibits a steeper precision drop-off, highlighting the value of nonlinear ensemble methods for this dataset.

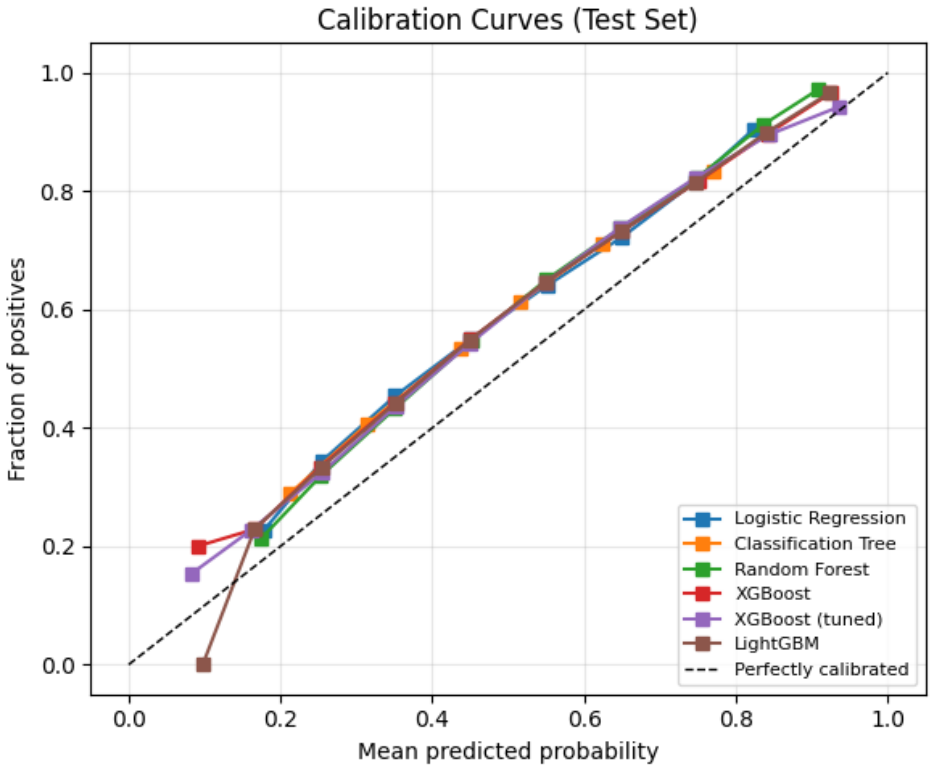

Figure 5: Calibration curves for all evaluated models on the test set. The dashed diagonal line represents perfect calibration. Most models follow the diagonal closely, indicating that predicted probabilities align well with observed reorder frequencies.

In addition to discrimination metrics, we evaluate probability calibration. Most models track the diagonal closely, indicating that predicted probabilities align well with empirical reorder frequencies. The tuned XGBoost model maintains strong calibration across probability bins, suggesting that its predicted probabilities can be interpreted reliably for ranking and recommendation applications.

6 Conclusion

By formulating the market basket analysis problem strictly as a supervised learning task, we developed a predictive model for estimating user reorder probabilities. Through careful data preprocessing, feature engineering, and the introduction of a sentinel value (`-9`) for previously unseen user-product interactions, we constructed a structured modeling array that captures both historical purchasing behavior and contextual ordering information.

To identify the most effective predictive approach, we evaluated a progression of models ranging from a baseline Logistic Regression to tree-based ensemble methods including Random Forest, XGBoost, and LightGBM. Hyperopt-based Bayesian optimization further improved the performance of the gradient boosting models.

Empirical results from the holdout test set and 5-fold cross-validation show that ensemble boosting methods consistently outperform simpler baselines. In particular, tuned XGBoost achieves the highest PR-AUC and ROC-AUC while maintaining stable performance across folds. These results

highlight the importance of nonlinear modeling approaches for capturing the interactions between user habits, product characteristics, and temporal shopping patterns.

From a practical perspective, such predictive systems can be integrated into online grocery platforms to generate personalized product recommendations. By identifying items that a user is most likely to reorder, retailers can improve customer convenience while increasing engagement and sales through targeted recommendations.

Future work could extend this framework by incorporating sequence-based models that capture the temporal ordering of user purchases or by integrating product co-purchase relationships through graph-based representations. These directions may further improve predictive accuracy and provide deeper insight into consumer purchasing behavior.

A Supplemental EDA

This appendix collects exploratory plots that support the feature rationale in Section 3 without expanding the main report with notebook-style intermediate outputs.

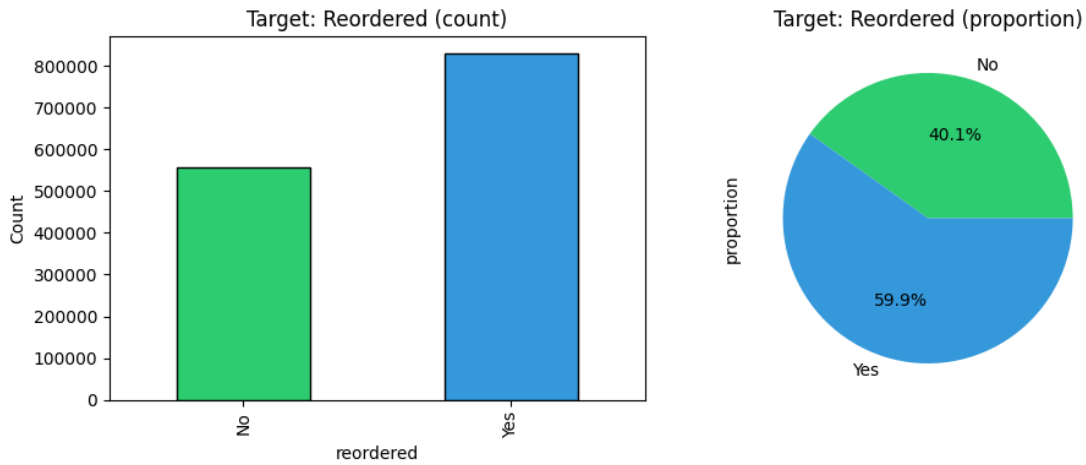

Figure A1: Target distribution for the `reordered` response, shown as both counts and class proportions.

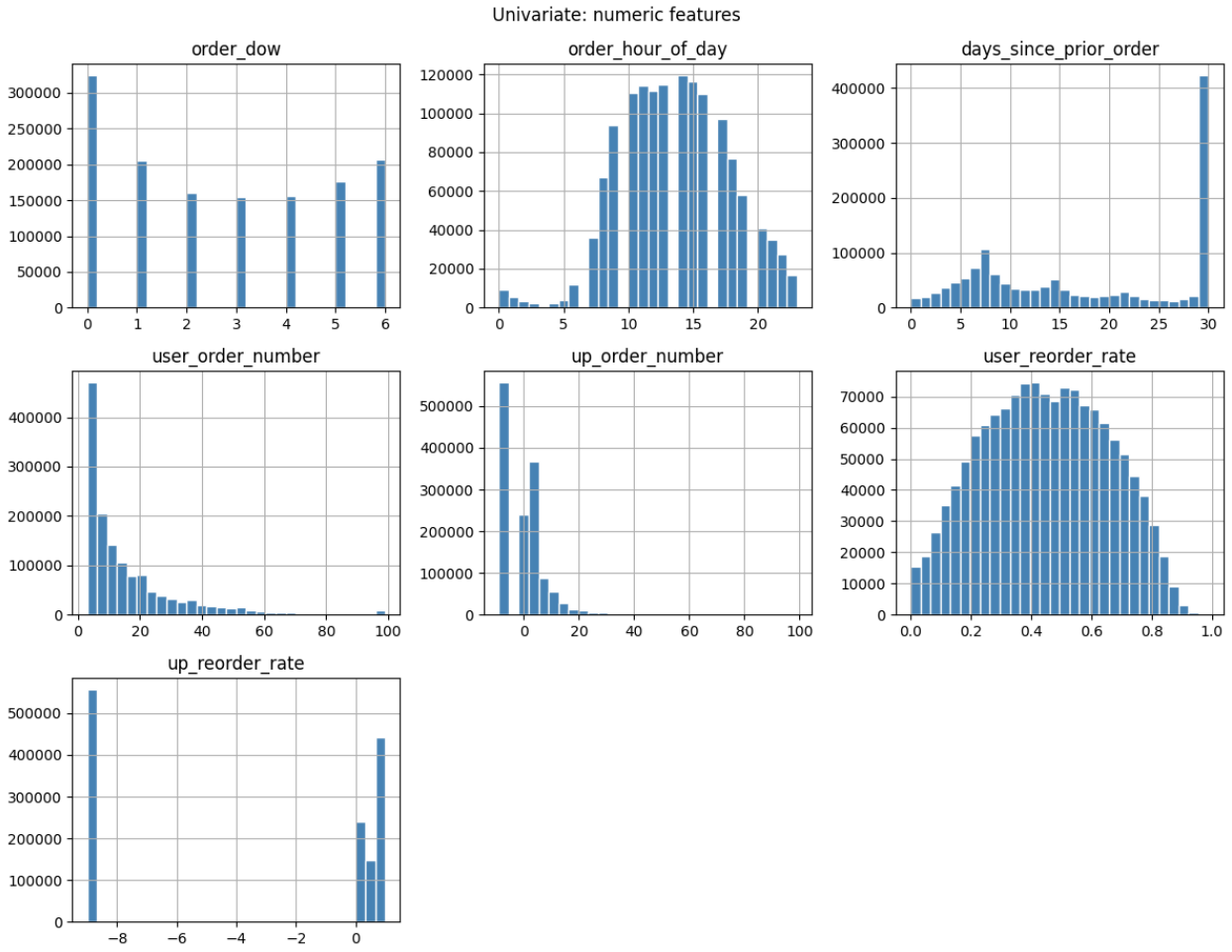

Figure A2: Univariate numeric feature distributions used to check scale, skew, and sampling coverage before model training.

B Model Interpretability

These plots complement the pruned tree visualization by showing which engineered signals contribute most strongly to model decisions. Together they reinforce that reorder behavior is driven primarily by prior user-product behavior, with recency and user-level reorder history providing secondary signal.

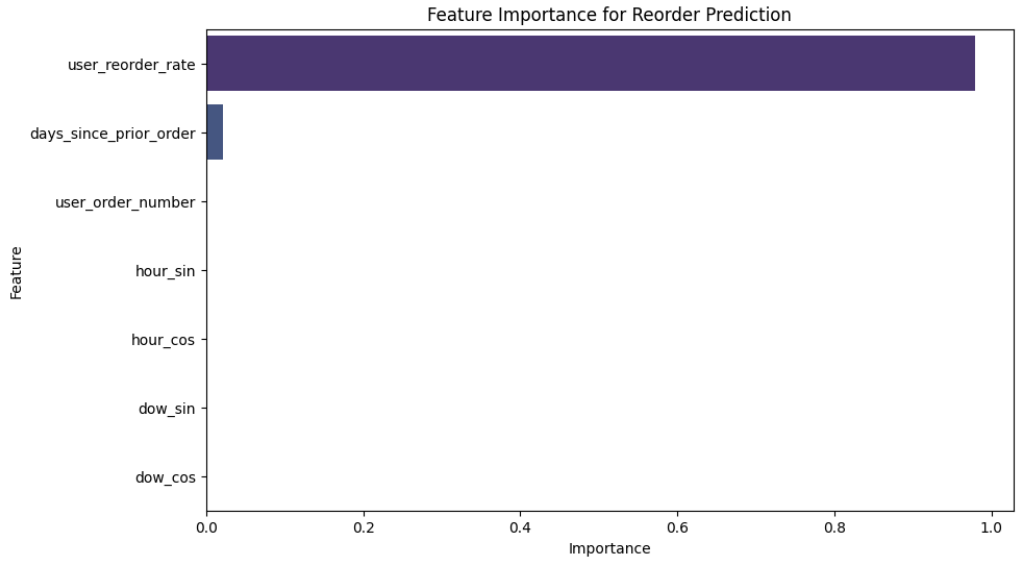

Figure B1: Decision-tree feature importance derived from the pruned classification tree.

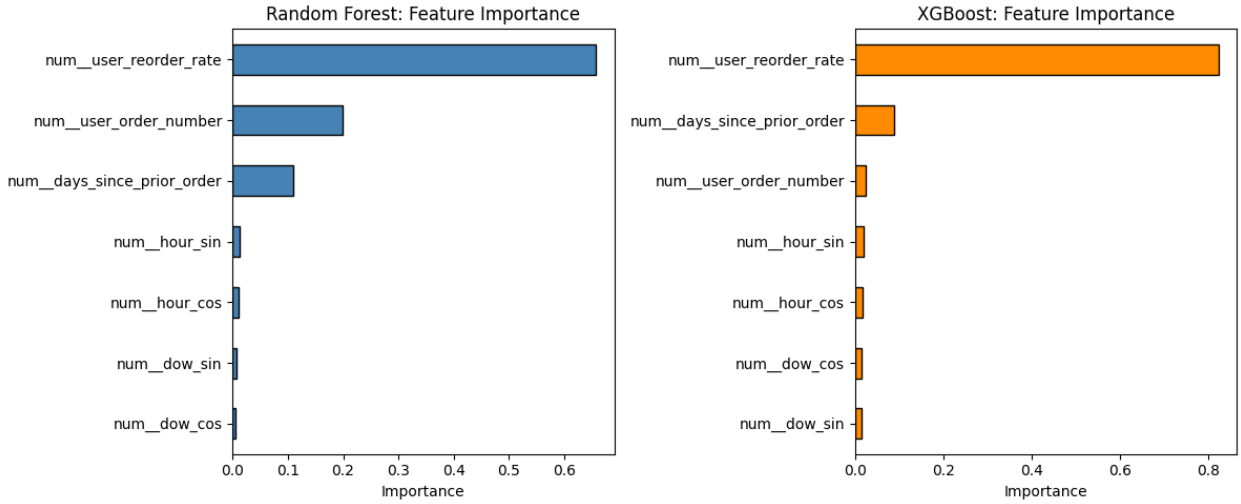

Figure B2: Feature importance comparison for Random Forest and XGBoost. Both models assign the strongest importance to user reorder rate, with days since prior order and user order number contributing additional signal.

C Additional Evaluation

This appendix gathers evaluation artifacts that were useful during model selection but were too detailed for the main report body.

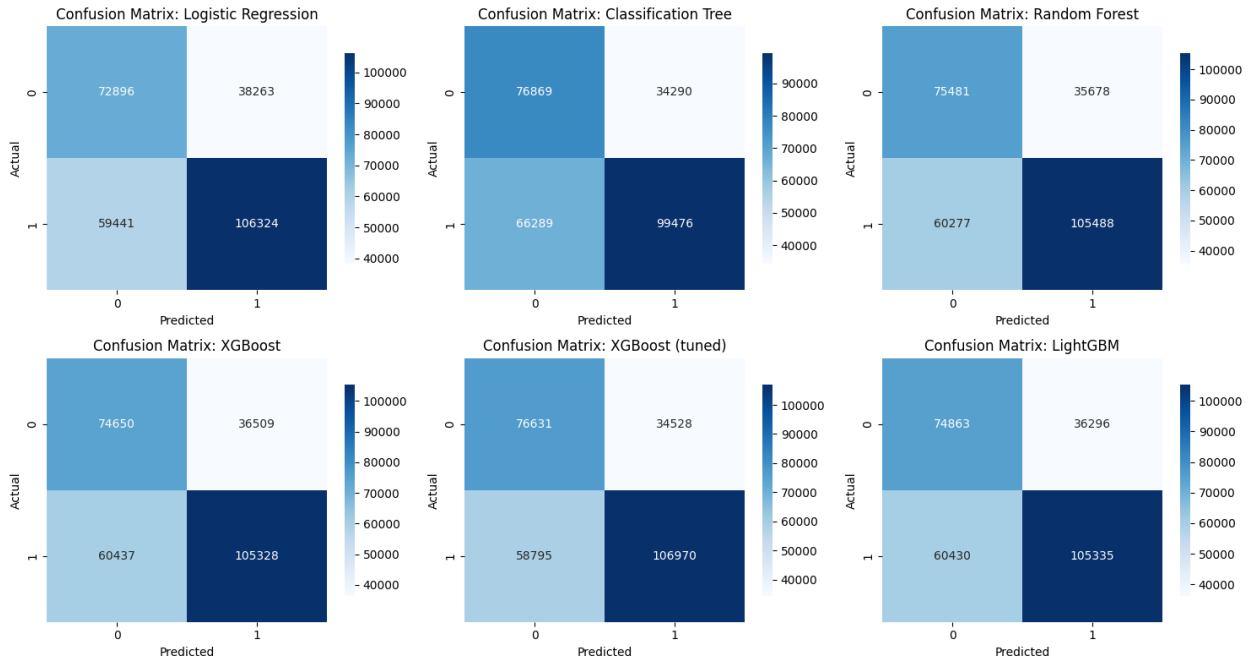

Figure C1: Confusion matrix heatmaps for all evaluated models using a 0.5 decision threshold on the test set.

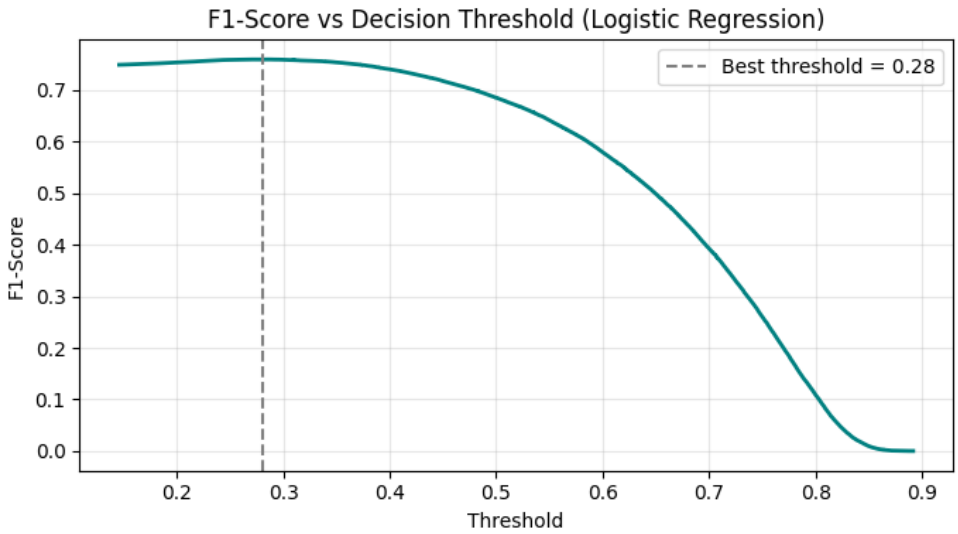

Figure C2: F1 score versus decision threshold for Logistic Regression. The best threshold occurs near 0.28.

Table C1: Single-split versus cross-validation summary

Model	Single ROC-AUC	CV ROC-AUC	Single PR-AUC	CV PR-AUC
Tuned XGBoost	0.7293	0.7291	0.7960	0.7959
Random Forest	0.7158	0.7156	0.7830	0.7824
LightGBM	0.7125	0.7122	0.7810	0.7805
XGBoost	0.7114	0.7113	0.7797	0.7793
Logistic Regression	0.7039	0.7037	0.7714	0.7706
Classification Tree	0.6919	0.6947	0.7349	0.7388

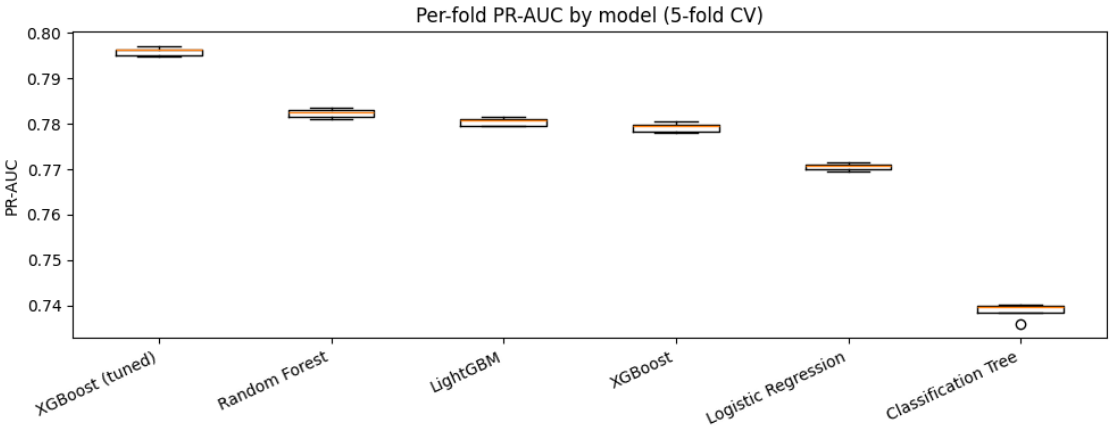

Figure C3: Per-fold PR-AUC distributions across the 5-fold cross-validation runs.

The notebook reports a paired t-test across the 5 CV folds comparing tuned XGBoost and Random Forest with a p-value of 0.0000 after rounding to four decimals, indicating a statistically significant improvement for tuned XGBoost on PR-AUC.

D Code References

Kaggle dataset link: Instacart Market Basket Analysis dataset

Notebook section mapping:

- **EDA and visual exploration**: `1. EDA` and `5. Visualizations`
- **Modeling array construction**: `4.1 Reshape Data` and `4.2 Feature Engineering`
- **Model training**: `6.4 Baseline - Logistic Regression` through `6.8` ensemble model sections
- **Evaluation and validation**: notebook Sections `6.9-6.11`